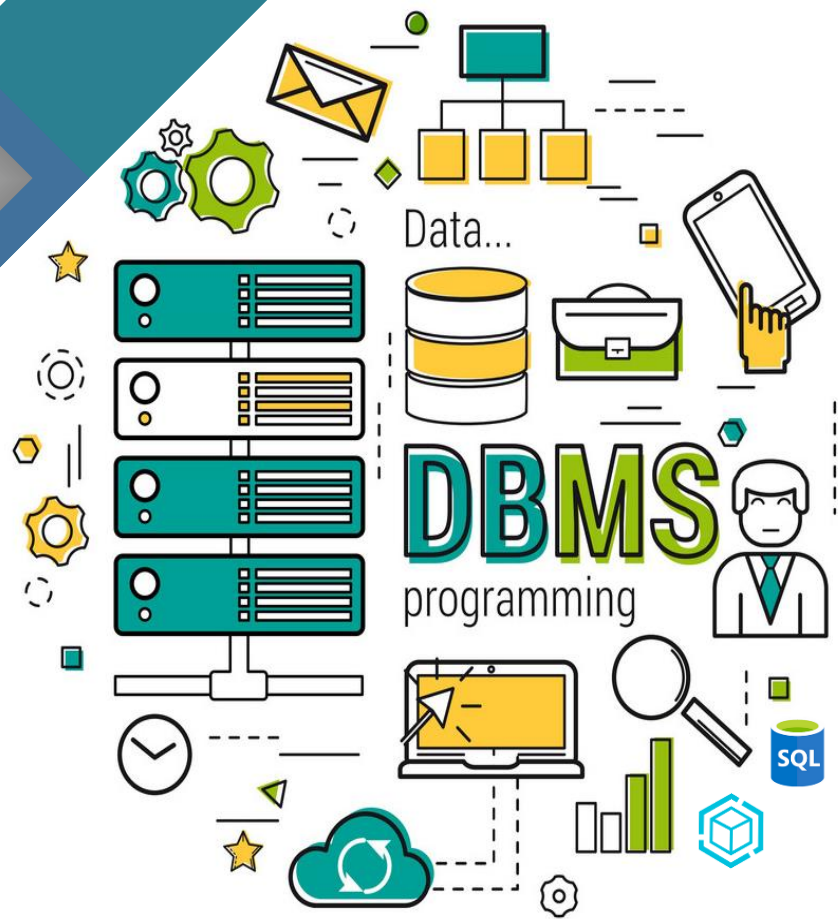




LJ University
University with a Difference



Unit-9 PL/SQL Concepts

DATABASE MANAGEMENT SYSTEM



Unit 9	PL/SQL Concepts
---------------	------------------------

Content

- **Introduction**
- **PL/pgSQL Block Structure**
- **PL/pgSQL Sub-blocks**
- **Variables in PL/pgSQL**
- **PostgreSQL SELECT INTO Statement**
- **Control Structures**
 - PL/pgSQL IF Statement
 - PL/pgSQL Loop Statement
 1. PL/pgSQL While Loop
 2. PL/pgSQL FOR Loop
- **PL/pgSQL Cursor**
- **Implicit Cursors**
- **Explicit Cursors**
- **Stored Function**
- **Stored Procedure**
- **Database Triggers**



Introduction

- PL/pgSQL is a **procedural programming language** for the PostgreSQL database system.
- PL/pgSQL allows you to extend the functionality of the PostgreSQL database server by creating server objects with complex logic.
- PL/pgSQL was designed to:
 1. Create user-defined functions, stored procedures, and triggers.
 2. Extend standard SQL by adding control structures such as if, case, and loop statements.
 3. Inherit all user-defined functions, operators, and types.

Advantages

- SQL is a query language that allows you to query data from the database easily. However, PostgreSQL only can execute SQL statements individually.
- It means that you have multiple statements, you need to execute them one by one like this:
 - First, send a query to the PostgreSQL database server.
 - Next, wait for it to process.
 - Then, process the result set.
 - After that, do some calculations.
 - Finally, send another query to the PostgreSQL database server and repeat this process.
- This process incurs the inter-process communication and network overheads. To resolve this issue, PostgreSQL uses PL/pgSQL.
- PL/pgSQL wraps multiple statements in an object and store it on the PostgreSQL database server. So instead of sending multiple statements to the server one by one, you can send one statement to execute the object stored in the server. This allows you to:
 - Reduce the number of round trips between the application and the PostgreSQL database server.
 - Avoid transferring the immediate results between the application and the server.

Disadvantages

- Slower in software development because PL/pgSQL requires specialized skills that many developers do not possess.
- Difficult to manage versions and hard to debug.
- May not be portable to other database management systems.

Dollar-Quoted String Constants

- The following shows the syntax of the dollar-quoted string constants:

\$tag\$ <string_constant> \$tag\$

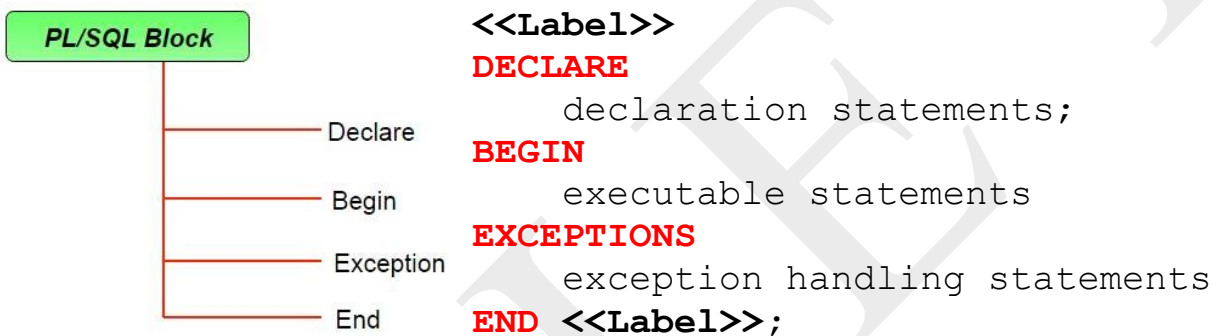
- In this syntax, the tag is optional. It may contain zero or many characters.
- Optionally can have a non-empty tag as part of the quote.

Example: \$_\$, \$abc\$

- It Can be used to prevent unnecessary escape characters throughout the string

PL/pgSQL Block Structure

- PL/pgSQL is a block-structured language, therefore, a PL/pgSQL function or stored procedure is organized into blocks.
- Syntax of a complete block in PL/pgSQL:



- Each block has two sections:** declaration and body. The declaration section is **optional** while the body section is **required**. A block is ended with a semicolon (;) after the END keyword.
- A block may have an optional label located at the beginning and at the end. You use the block label when you want to specify it in the EXIT statement of the block body or when you want to qualify the names of variables declared in the block.
- The declaration section is where you declare all variables used within the body section. Each statement in the declaration section is terminated with a semicolon (;).
- The body section is where you place the code. Each statement in the body section is also terminated with a semicolon (;).
- All PL/pgSQL code is composed of a single block or blocks that occur either sequentially or nested within another block. There are two kinds of blocks:

Anonymous blocks (DO)

- Generally constructed dynamically and executed only once by the user. It is sort of a complex SQL statement.

Named blocks (Functions and Stored Procedures)

- Have a name associated with them, are stored in the database, and can be executed repeatedly, and can take in parameters.

PL/pgSQL Block Structure Example

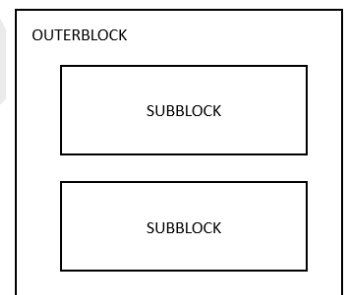
```
Do $$
declare
a integer;
b integer;
c integer;
begin
    a:=20;
    b:=30;
    c:=a+b;
    Raise notice 'a,b,c=%,%,%', a, b, c;
end;
$$
```

Data Output	Messages	Notifications
	NOTICE: a,b,c=20,30,50	DO

- Notice that the DO statement does not belong to the block. It is used to execute an anonymous block. PostgreSQL introduced the DO statement since version 9.0.

PL/pgSQL Sub-blocks

- PL/pgSQL allows you to place a block inside the body of another block. The block nested inside another block is called a subblock. The block that contains the subblock is referred to as an outer block. The following picture illustrates the outer block and subblock:
- Typically, you divide a large block into smaller and more logical subblocks. The variables in the subblock can have the names as the ones in the outer block, even though it is not a good practice.



Variables in PL/pgSQL

- A variable is a **meaningful name of a memory location**. A variable holds a value that can be changed through the block. A variable is always **associated with** a particular **data type**. Before using a variable, you **must declare it in the declaration section** of the PL/pgSQL block.

Syntax:

variable_name data_type [:= expression] ;

In this syntax:

- First, specify the name of the variable. It is a good practice to assign a meaningful name to a variable. For example, instead of naming a variable you should use index or counter.
- Second, associate a specific data type with the variable. The data type can be any valid data type such as integer, numeric, varchar, and char.
- Third, **optionally assign** a default value to a variable. If you don't do so, the initial value of the variable is NULL.

Note that you can use either **:=** or **=** assignment operator to initialize and assign a value to a variable.

```
do $$  
declare  
    counter    integer := 1;  
    first_name varchar(50) := 'John';  
    last_name  varchar(50) := 'Doe';  
    payment    numeric(11,2) := 20.5;  
begin  
    raise notice '% % % has been paid % USD', counter,  
    first_name, last_name, payment;  
end $$;
```

Data Output Messages Notifications

NOTICE: 1 John Doe has been paid 20.50 USD
DO

➤ Copying data types

- The **%type** provides the data type of a table column or another variable. Typically, you use the **%type** to declare a variable that holds a value from the database or another variable.
- The following illustrates how to declare a variable with the data type of a table column:

```
variable_name table_name.column_name%type;
```

➤ PL/pgSQL Row Types

- To store the whole row of a result set returned by the select into statement, you use the row-type variable or row variable.

```
row_variable table_name%ROWTYPE;  
row_variable view_name%ROWTYPE;
```

- To access the individual field of the row variable, you use the dot notation (.) like this:

```
row_variable.field_name
```

PostgreSQL SELECT INTO Statement

- The select into statement allows you to select data from the database and assign the data to a variable.

Syntax:

```
SELECT select_list INTO variable_name FROM table_name  
WHERE condition;
```

- In this syntax, you place the variable after the into keyword. The select into statement will assign the data returned by the select clause to the variable.
- Besides selecting data from a table, you can use other clauses of the select statement such as **JOIN**, **GROUP BY**, and **HAVING**.

⇒ PL/pgSQL SELECT INTO Example

- We will use the film table from the dvdrental for the demonstration.

```
DO $$
```

```
declare
```

```
    actor_count integer;
```

```
begin
```

```
-- Select the number of actors from the actor table
```

```
SELECT count(*) INTO actor_count FROM actor;
```

```
-- Select the number of actors
```

```
raise notice 'The number of actors: %', actor_count;
```

```
end; $$
```

Data Output	Messages	Notifications
		NOTICE: The number of actors: 200 DO

- First, declare a **variable** called **actor_count** that stores the number of actors from the actor table. Second, use the **select into** statement to assign the number of actors to the **actor_count**. Finally, display a message that shows the **value** of the **actor_count** variable using the **raise notice** statement.

⇒ PL/pgSQL Row Types Example

- We'll use the actor table from the dvdrental to show how row types work:
- The following example shows the first name and last name of the actor id 10:

actor
* actor_id
first_name
last_name
last_update

```
DO $$
```

```
declare
```

```
    selected_actor actor%rowtype;
```

```
begin
```

```
-- select actor with id 10
```

```
SELECT * INTO selected_actor FROM actor
```

```
WHERE actor_id = 10;
```

```
-- show the number of actor
```

```
raise notice 'The actor name is % %',
```

```
    selected_actor.first_name, selected_actor.last_name;
```

```
end;
```

```
$$
```

Data Output	Messages	Notifications
		NOTICE: The actor name is Christian Gable DO

How it works.

- First, declare a row variable called **selected_actor** whose **datatype is the same as the row in the actor table**.
- Second, assign the row whose value in the **actor_id** column is 10 to the **selected_actor variable** by using the select into statement.
- Third, show the **first name and last name** of the selected actor by using the raise notice statement. It accessed the first_name and last_name fields using the dot notation.

Control Structures

PL/pgSQL IF Statement

- The IF statement determines which statements to execute based on the result of a Boolean expression. PL/pgSQL provides you with three forms of the IF statements.
 - [IF THEN](#)
 - [IF THEN ELSE](#)
 - [IF THEN ELSIF](#)

1) PL/pgSQL IF-THEN Statement

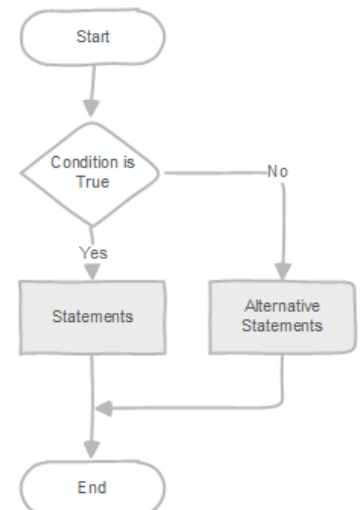
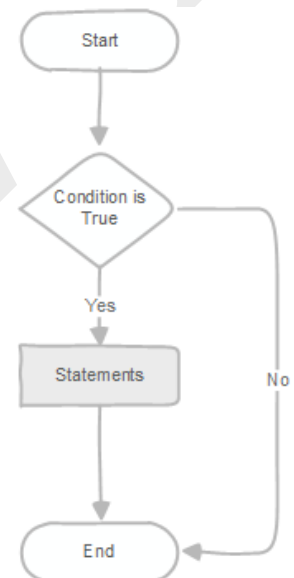
Syntax:

```
if condition then
  statements;
end if;
```

- The IF statement executes statements if a condition is true. If the condition evaluates to false, the control is passed to the next statement after the END if part.
- The condition is a boolean expression that evaluates to true or false. The statements can be one or more statements that will be executed if the condition is true.
- It can be any valid statement, even another if statement. When an IF statement is placed inside another if statement, it is called a NESTED-IF statement.
- Refer following Example.

```
Do $$
declare
counter int:=1;
begin
  if counter=1 then
    raise notice 'counter is ok';
  end if;
end; $$
```

Data Output	Messages	Notifications
	NOTICE: counter is ok	DO



2) PL/pgSQL IF-THEN-ELSE Statement

Syntax:

```
if condition then
    statements;
else
    alternative-statements;
end if;
```

- The if then else statement executes the statements in the if branch if the condition evaluates to true; otherwise, it executes the statements in the else branch.
- Refer following Example.

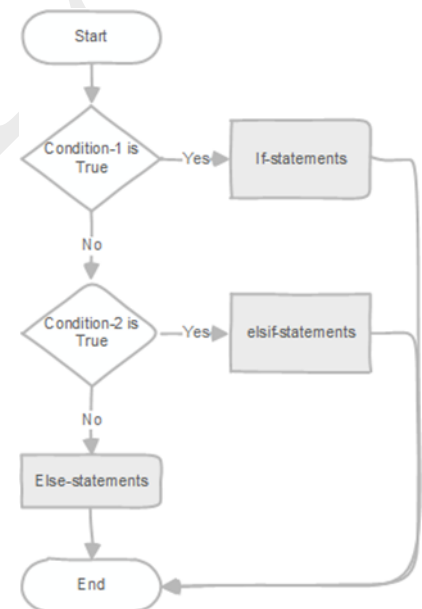
```
Do $$
declare
    counter int:=1;
begin
    if counter=2 then
        raise notice 'counter is ok';
    else
        raise notice 'counter is not ok';
    end if;
end;
$$
```

Data Output	Messages	Notifications
	NOTICE: counter is not ok	DO

3) PL/pgSQL IF-THEN-ELSIF Statement:

Syntax:

```
if condition_1 then
    statement_1;
elseif condition_2 then
    statement_2;
else
    else-statement;
end if;
```



- The if and if-then else statements evaluate one condition. However, the if then elseif statement evaluates multiple conditions.
- If a condition is true, the corresponding statement in that branch is executed. For example, if the condition_1 is true then the if then elseif executes the statement_1 and stops evaluating the other conditions.
- If all conditions evaluate to false, the if then elseif executes the statements in the else branch.

- Refer following Example.

```
Do $$
declare
    counter int:=1;
begin
    if counter=1 then
        raise notice 'counter is ok';
    elseif counter=2 then
        raise notice 'counter is good';
    else
        raise notice 'counter is not ok';
    end if;
end;
$$
```

Data Output	Messages	Notifications
	NOTICE: counter is ok	DO

Example based on IF Statement

Example 1 Find the maximum number using the if-then-elseif statement.

```
Do $$
declare
    a int:=5;
    b int:=6;
    c int:=2;
begin
    if a>b then
        if a>c then
            raise notice 'a is maximum';
        else
            raise notice 'c is maximum';
        end if;
    elseif b>c then
        raise notice 'b is maximum';
    else
        raise notice 'c is maximum';
    end if;
end;
$$
```

Data Output	Messages	Notifications
	NOTICE: b is maximum	DO

Example 2 Find out the given number is even or odd using IF statement.

```
Do $$
declare
a int:=5;
begin
    if a%2=0 then
        raise notice 'a is even';
    else
        raise notice 'a is odd';
    end if;
end;
$$
```

Data Output	Messages	Notifications
	NOTICE: a is odd	DO

Example 3 Create Following table and update the name to ajay where age is 30 using if statement.

```
CREATE TABLE student (name varchar (10), age int);
Insert Into student values ('abc', 20), ('def', 30);
```

```
Do $$
declare
n student.name%type;
a student.age%type:= 30;
begin
    n:='ajay';
    if a=30 then
        update public.student set name = n WHERE age = a;
        raise notice 'updation done';
    else
        raise notice 'updation not done';
    end if;
end;
Select * from student;
```

	name character varying (10)	age integer
1	abc	20
2	def	30

	name character varying (10)	age integer
1	abc	20
2	ajay	30

PL/pgSQL Loop Statement

- The loop defines an unconditional loop that executes a block of code repeatedly until terminated by an exit or return statement.

Syntax:

```
<<label>>  
loop  
    statements;  
end loop;
```

- Typically, you use an if statement inside the loop to terminate it based on a condition like this:

```
<<label>>  
loop  
    statements;  
    if condition then  
        exit;  
    end if;  
end loop;
```

- It's possible to place a loop statement inside another loop statement. When a loop statement is placed inside another loop statement, it is called a NESTED LOOP.

```
<<outer>>  
loop  
    statements;  
    <<inner>>  
    loop  
        /* ... */  
        exit <<inner>>;  
    end loop;  
end loop;
```

- When you have nested loops, you need to use the loop label so that you can specify it in the exit and continue statement to indicate which loop these statements refer to.

PL/pgSQL Exit statement

- The exit statement allows you to terminate a loop including an unconditional loop, a while loop, and a for loop.

Syntax:

```
exit [label] [when boolean_expression];
```

- The label is the loop label of the current loop where the exit is in or the loop label of the outer loop. Depending on the label, the exit statement will terminate the corresponding loop. If you don't use the label, the exit statement will terminate the current loop.
- The when boolean_expression clause is used to specify a condition that terminates a loop. The exit statement will terminate the loop if the boolean_expression evaluates to true.
- The following statements are equivalent:

```
exit when counter > 10;
```

or

```
if counter > 10 then  
    exit;  
end if;
```

- The exit when is definitely cleaner and shorter.
- In addition to terminating a loop, you can use the exit statement to terminate a block specified by the begin...end keywords. In this case, the control is passed to the statement after the end keyword of the current block:

```
<<block_label>>
```

```
BEGIN
```

```
    -- some code
```

```
    EXIT [block_label] [WHEN condition];
```

```
    -- some more code
```

```
END block_label;
```

Example 1 Display square of numbers from 1 to 10 using EXIT statement.

```
Do $$
declare
    counter integer:=1;
begin
loop
    exit when counter>10;
    raise notice 'counter is %', counter*counter;
    counter=counter+1;
end loop;
end; $$
```

Data Output	Messages	Notifications
NOTICE:	counter is 1	
NOTICE:	counter is 4	
NOTICE:	counter is 9	
NOTICE:	counter is 16	
NOTICE:	counter is 25	
NOTICE:	counter is 36	
NOTICE:	counter is 49	
NOTICE:	counter is 64	
NOTICE:	counter is 81	
NOTICE:	counter is 100	
DO		

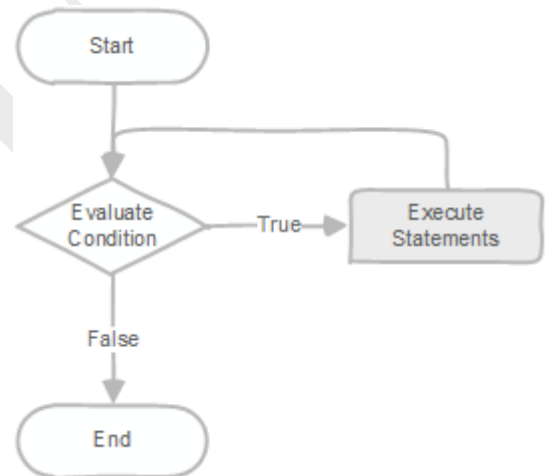
PL/pgSQL While Loop

The while loop statement executes a block of code until a condition evaluates to false.

Syntax:

```
while condition loop
    statements;
end loop;
```

- In this syntax, PostgreSQL evaluates the condition before executing the statements.
- If the condition is true, it executes the statements. After each iteration, the while loop evaluates the condition again. Inside the body of the while loop, you need to change the values of some variables to make the condition false or null at some points.
- Otherwise, you will have an indefinite loop. Because the while loop tests the condition before executing the statements, the while loop is sometimes referred to as a **pretest loop**.



How it works.

```
Do $$
declare
    counter integer := 0;
begin
    while counter < 5 loop
        raise notice 'Counter %', counter;
        counter := counter + 1;
    end loop;
end; $$
```

Data Output	Messages
NOTICE:	Counter 0
NOTICE:	Counter 1
NOTICE:	Counter 2
NOTICE:	Counter 3
NOTICE:	Counter 4
DO	

- First, declare the counter variable and initialize its value to 0.
- Second, use the while loop statement to show the current value of the counter as long as it is less than 5. In each iteration, increase the value of counter by one. After 5 iterations, the counter is 5 therefore the while loop is terminated.

Example 1 Display square of numbers from 1 to 10 using WHILE loop.

```
Do $$
declare
    counter integer:=1;
begin
while counter<11 loop
    raise notice 'counter is %', counter=counter+1;
end loop;
end;
$$
```

Data Output	Messages
NOTICE:	counter is 1
NOTICE:	counter is 4
NOTICE:	counter is 9
NOTICE:	counter is 16
NOTICE:	counter is 25
NOTICE:	counter is 36
NOTICE:	counter is 49
NOTICE:	counter is 64
NOTICE:	counter is 81
NOTICE:	counter is 100
DO	

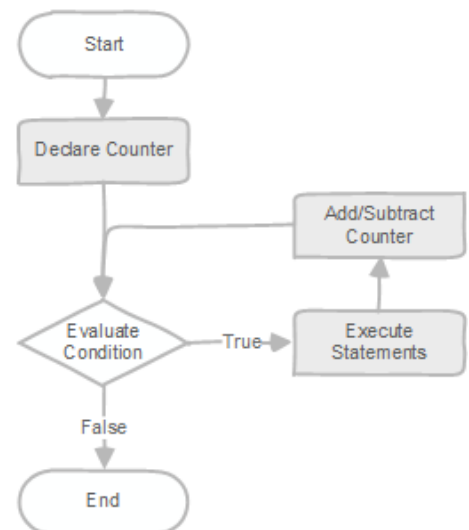
PL/pgSQL FOR Loop

- Using PL/pgSQL FOR loop to iterate over a range of integers

Syntax:

```
[ <<label>> ]
for loop_counter in [reverse] from.. to [by step] loop
    statements
end loop [ label ];
```

- First, the for loop creates an integer variable loop_counter which is accessible inside the loop only. By default, the for loop adds the step to the loop_counter after each iteration. However, when you use the reverse option, the for loop subtracts the step from loop_counter.
- Second, the from and to are expressions that specify the lower and upper bound of the range. The for loop evaluates these expressions before entering the loop.
- Third, the step that follows the by keyword specifies the iteration step. It defaults to 1. The for loop evaluates this step expression once only.





Example 1 Display square of numbers from 1 to 10 using FOR Loop.

```
Do $$
begin
for counter in 1..10 loop
    raise notice 'counter is %',counter*counter;
end loop;
end;
$$
```

Example 2 Display square of numbers from 1 to 10 of odd numbers using FOR Loop.

```
Do $$
begin
for counter in 1..10 by 2 loop
    raise notice 'counter is %',
        counter*counter;
end loop;
end;
$$
```

Data Output	Messages
NOTICE:	counter is 1
NOTICE:	counter is 9
NOTICE:	counter is 25
NOTICE:	counter is 49
NOTICE:	counter is 81
DO	

Example 3 Display square of numbers from 10 to 1.

```
Do $$
begin
for counter in reverse 10..1 loop
    raise notice 'counter is %',
        counter*counter;
end loop;
end;
$$
```

Data Output	Messages
NOTICE:	counter is 100
NOTICE:	counter is 81
NOTICE:	counter is 64
NOTICE:	counter is 49
NOTICE:	counter is 36
NOTICE:	counter is 25
NOTICE:	counter is 16
NOTICE:	counter is 9
NOTICE:	counter is 4
NOTICE:	counter is 1
DO	

Example 4 Display square of numbers from 10 to 1 of even numbers:

```
Do $$
begin
for counter in reverse 10..1 by 2 loop
    raise notice 'counter is %',
        counter*counter;
end loop;
end;
$$
```

Data Output	Messages
NOTICE:	counter is 100
NOTICE:	counter is 64
NOTICE:	counter is 36
NOTICE:	counter is 16
NOTICE:	counter is 4
DO	



PL/pgSQL Cursor

- PL/pgSQL creates a memory area, known as the context area, for processing an SQL statement, which contains all the information needed for processing the statement; for example, **the number of rows processed**, etc.
- A cursor is a pointer to this context area. PL/pgSQL controls the context area through a cursor. **A cursor holds the rows (one or more) returned by a SQL statement.** The set of rows the cursor holds is referred to as the **active set**.
- You can name a cursor so that it could be referred to in a program to fetch and process the rows returned by the SQL statement, one at a time. There are two types of cursors.
 1. **Implicit cursors**
 2. **Explicit cursors**

1. Implicit cursors

- Implicit cursors are automatically created by PL/pgSQL whenever an SQL statement is executed, when there is no explicit cursor for the statement.
- Programmers cannot control the implicit cursors and the information in it.
- Whenever a DML statement (INSERT, UPDATE and DELETE) is issued, an implicit cursor is associated with this statement.
- For INSERT operations, the cursor holds the data that needs to be inserted.
- For UPDATE and DELETE operations, the cursor identifies the rows that would be affected.
- In PL/pgSQL, you can refer to the most recent implicit cursor as the SQL cursor, which always has attributes such as FOUND, NOT FOUND, and ROW_COUNT.

Attribute	Description
FOUND	Its return value is TRUE if DML statements like INSERT, DELETE and UPDATE affect at least one row or more rows or a SELECT INTO statement returned one or more rows. Otherwise, it returns FALSE.
NOT FOUND	Its return value is TRUE if DML statements like INSERT, DELETE and UPDATE affect no row, or a SELECT INTO statement return no rows. Otherwise, it returns FALSE. It is a just opposite of FOUND.
ROW_COUNT	It returns the number of rows affected by DML statements like INSERT, DELETE, and UPDATE or returned by a SELECT INTO statement.
Note: Above mentioned attributes are used with explicit cursor also.	

Example

```
CREATE TABLE Employee
(id int, name varchar (20), age int);
INSERT INTO Employee VALUES
(1, 'Jay', 60), (2, 'Ajay', 35), (3, 'Vijay', 40);
```

Do \$\$

declare

```
row_counter int;
```

begin

```
Delete from Employee where age>50;
```

```
if found then
```

```
get diagnostics row_counter=row_count;
```

```
raise notice 'deleted rows are %',row_counter;
```

```
end if;
```

end; \$\$

	id [PK] integer	name character varying	age integer
1	1	Jay	60
2	2	Ajay	35
3	3	Vijay	40

NOTICE: deleted rows are 1
DO

2. Explicit Cursors

- The Explicit cursors are defined by the programmers to gain more control over the context area.
- These cursors should be defined in the declaration section of the PL/pgSQL block.
- It is created on a SELECT statement which returns more than one row.

➤ Steps:

- You must follow these steps while working with an explicit cursor.

★ Step 1:

- Declare the cursor to initialize in the memory.

★ Step 2:

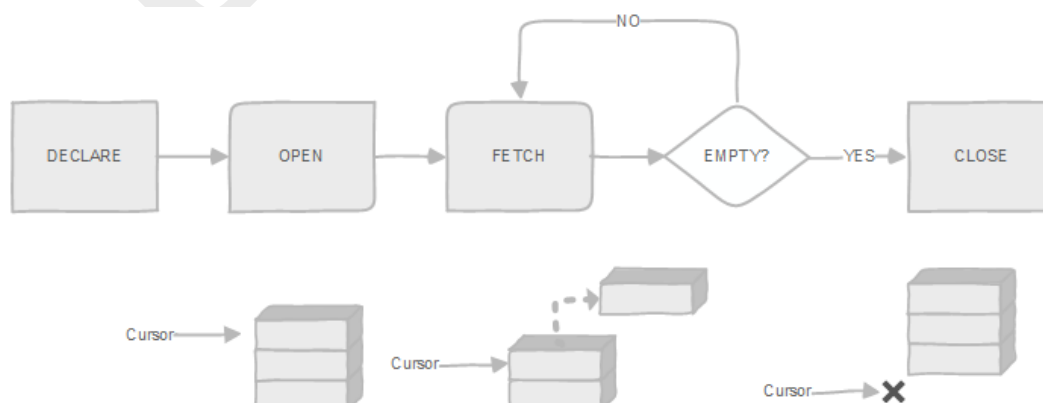
- Open the cursor to allocate memory.

★ Step 3:

- Fetch the cursor to retrieve data.

★ Step 4:

- Close the cursor to release allocated memory.



Example 1 To fetch record of column id, name & address into cursor & also display the same.

```
create table t1 (id integer,  
                name varchar (10), age int,  
                address varchar (100), salary int);  
insert into t1 values (1,'ram',30,'abad',25000);  
  
Do $$  
declare  
    cl_id t1.id%type;  
    cl_name t1.name%type;  
    cl_address t1.address%type;  
    t1_cursor cursor for select id,name, address from t1;  
begin  
    open t1_cursor;  
    loop  
        fetch t1_cursor into cl_id,cl_name, cl_address;  
        exit when not found;  
        raise notice 'id: %, Name: %, address: %',  
            cl_id,cl_name,cl_address;  
    end loop;  
    close t1_cursor;  
end;  
$$
```

NOTICE: id: 1, Name: ram, address: abad
DO

Example 2 Update balance of t1 table whose id is 1 and salary is 25000 to 25500 using cursor.

First create table and insert values.

```
create table t1 (id integer primary key,  
                name varchar (10), age int,  
                address varchar (100), salary int);  
insert into t1 values  
    (1,'Amit',30,'Abad',25000),  
    (2,'Sumit',32,'Brd',22000),  
    (3,'Mit',33,'Srt',20000);
```

	id [PK] integer	name character varying	age integer	address character varying	salary integer
1	1	Amit	30	Ahmedabad	25000
2	2	Sumit	32	Baroda	22000
3	3	Mit	33	Surat	20000

Now update with cursor

```

Do $$
declare
    c1_id t1.id%type;
    c2_sal t1.salary%type;
    Updated_salary cursor for Select id, salary from t1;
begin
    open updated_salary;
    loop
        fetch Updated_salary into c1_id, c2_sal;
        exit when not found;
        if c1_id=1 then
            update t1 set salary=salary+500 where c1_id=id;
-- update t1 set salary=salary+500 where CURRENT of
Updated_salary; (also use this statement to update salary)
        end if;
    end loop;
    close Updated_salary;
end;
$$

```

	id [PK] integer	name character varying	age integer	address character varying	salary integer
1	2	Sumit	32	Baroda	22000
2	3	Mit	33	Surat	20000
3	1	Amit	30	Ahmedabad	25500

Example 3 Write a PL/pgSQL program using explicit cursor for following statement.

Delete row from table account where branch is Jivraj Park & insert into table named New_Branch.

First Create two tables named account (with values) and New_Branch;

```

CREATE TABLE account (ano int, balance numeric,
                        branch varchar(50));

```

```

INSERT INTO account
VALUES (101,1000,'Jivraj Park'),
(102,2000,'Vastrapur'),
(103,3000,'Bopal');

```

	ano integer	balance numeric	branch character varying
1	101	1000	Jivraj Park
2	102	2000	Vastrapur
3	103	3000	Bopal

```

Create table New_Branch (ano int,
balance numeric);

```

	ano integer	balance numeric
--	----------------	--------------------

```
Do $$
declare
    c1_ano account.ano%type;
    c1_balance account.balance%type;
    c1_branch account.branch%type;
    Update_Branch cursor for select ano, balance, branch
    from account;
begin
    open Update_Branch;
loop
    fetch Update_Branch into c1_ano, c1_balance,
    c1_branch;
    exit when not found;
    if c1_branch = 'Jivraj Park' then
        insert into New_Branch values (c1_ano,c1_balance);
        delete from account where ano = c1_ano;
    end if;
end loop;
close Update_Branch;
end;
$$
```

Select * From New_Branch;

	ano integer	balance numeric
1	101	1000

Stored Function

- The create function statement allows you to define a new user-defined function.

Syntax:

```
CREATE[or replace]FUNCTION function_name(parameter_list)
    RETURNS return_datatype
    language plpgsql
    as
    $$
declare
    -- variable declaration
begin
    -- logic
end;
$$
```

In this syntax:

- First, specify the name of the function after the create function keywords. If you want to replace the existing function, you can use the or replace keywords.
- Then, specify the function parameter list surrounded by parentheses after the function name. A function can have zero or many parameters.
- Next, specify the datatype of the returned value after the returns keyword.
- After that, use the language plpgsql to specify the procedural language of the function. Note that PostgreSQL supports many procedural languages, not just plpgsql.
- Finally, place a block in the dollar-quoted string constant.

Example 1 Creates a function that counts the films whose length between the len_from and len_to parameters (refer dvdrental)

```
CREATE or replace FUNCTION get_film_count
(len_from int, len_to int)
RETURNS integer
language plpgsql as
$$
declare
    film_count integer;
begin
    SELECT count(*) INTO film_count FROM film
    WHERE length BETWEEN len_from AND len_to;
    return film_count;
end;
$$;
```

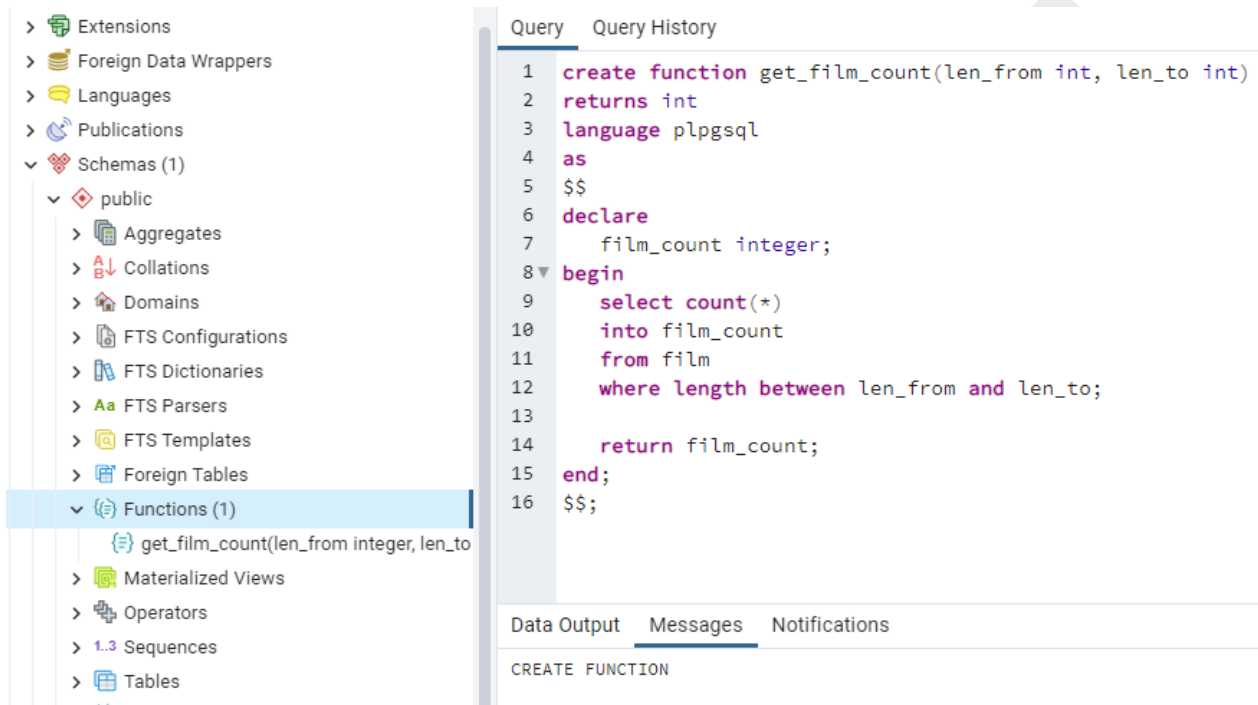
In the header section:

- First, the name of the function is get_film_count that follows the create function keywords.
- Second, the get_film_count() function accepts two parameters len_from and len_to with the integer datatype.
- Third, the get_film_count function returns an integer specified by the returns int clause.
- Finally, the language of the function is plpgsql indicated by the language plpgsql.

In the function body:

- Use the dollar-quoted string constant syntax that starts with \$\$ and ends with \$\$.
- Between these \$\$, you can place a block that contains the declaration and logic of the function.

- In the declaration section, declare a variable called `film_count` that stores the number of films selected from the `film` table.
- In the body of the block, use the `select into` statement to select the number of films whose length are between `len_from` and `len_to` and assign the result to the `film_count` variable. At the end of the block, use the `return` statement to return the `film_count`.
- To execute the create function statement, you can use any PostgreSQL client tool including `psql` and `pgAdmin`.



The screenshot shows the pgAdmin interface. On the left, the 'Schemas (1)' tree is expanded to 'public', and 'Functions (1)' is selected, showing the function `get_film_count(len_from integer, len_to integer)`. The main pane shows the SQL query editor with the following code:

```
1 create function get_film_count(len_from int, len_to int)
2 returns int
3 language plpgsql
4 as
5 $$
6 declare
7     film_count integer;
8 begin
9     select count(*)
10    into film_count
11   from film
12  where length between len_from and len_to;
13
14     return film_count;
15 end;
16 $$;
```

Below the query editor, the 'Messages' tab is active, showing the message 'CREATE FUNCTION'.

➤ Calling a user-defined function

- PostgreSQL provides you with three ways to call a user-defined function:
 1. Using positional notation
 2. Using named notation
 3. Using the mixed notation.

1) Using positional notation

- To call a function using the positional notation, you need to specify the arguments in the same order as parameters. For example:

```
select get_film_count (40,90);
```

Output

```
get_film_count
-----
              325
```

- In this example, the arguments of the `get_film_count()` are 40 and 90 that corresponding to the `from_len` and `to_len` parameters.
- You call a function using the positional notation when the function has few parameters.

2) Using named notation

- The following shows how to call the `get_film_count` function using the positional notation:

```
select get_film_count(len_from => 40, len_to => 90);
```

- For backward compatibility, PostgreSQL supports the older syntax based on `:=` as follows:

```
select get_film_count (len_from := 40, len_to := 90);
```

3) Using mixed notation

- The mixed notation is the combination of positional and named notations. For example:

```
select get_film_count (40, len_to => 90);
```

Example 2 Find out minimum value out of given two values using function:

```
create or replace function min (x int, y int)
returns integer as
$$
```

```
Declare
```

```
    z integer;
```

```
begin
```

```
    if x<y then
```

```
        z:=x;
```

```
    else
```

```
        z:=y;
```

```
    end if;
```

```
return z;
```

```
end;
```

```
$$
```

```
language plpgsql;
```

To call Function:

```
Select min (5,3);
```

	min integer	
1		3

Example 3 Create function using PL/pgSQL to change salary from 25000 to 30000 whose emp id is 1.

Let initial table emp has following details:

	empid integer	name character varying (20)	age integer	salary integer
1	1	Mahesh	25	25000

```
create or replace function calculate_salary
```

```
(id integer, amount integer) returns integer as $$
```

```
declare
```

```
    currentsal integer;
```

```
    newsal integer;
```

begin

```
SELECT salary INTO currentsal FROM emp WHERE empid=id;  
newsal:= currentsal+amount;  
UPDATE emp SET salary=newsal WHERE empid=id;  
return newsal;
```

end;

\$\$

language plpgsql;

Call the function:

```
select calculate_salary(1,5000);
```

Output

```
Select * from emp;
```

	empid integer	name character varying (20)	age integer	salary integer
1	1	Mahesh	25	30000

Stored Procedure

- A drawback of user-defined functions is that they cannot execute transactions. In other words, inside a user-defined function, you cannot start a transaction, and commit or rollback it.
- To define a new stored procedure, you use the create procedure statement.

Syntax:

```
create [or replace] procedure procedure_name  
(parameter_name [IN | INOUT] type [, ...])  
language plpgsql  
as $$  
declare  
    -- variable declaration  
begin  
    -- stored procedure body  
end; $$
```

- In this syntax:
 - First, specify the name of the stored procedure after the create procedure keywords.
 - Second, define parameters for the stored procedure. A stored procedure can accept zero or more parameters.



- Third, specify plpgsql as the procedural language for the stored procedure. Note that you can use other procedural languages for the stored procedure such as SQL, C, etc.
- Finally, use the dollar-quoted string constant syntax to define the body of the stored procedure.
- **Parameters in stored procedures can have the in and inout modes. They cannot have the out mode.**
- A stored procedure does not return a value. You cannot use the return statement with a value inside a store procedure
- However, you can use the return statement without the expression to stop the stored procedure immediately:

➤ **Calling a stored procedure**

- To call a stored procedure, you use the CALL statement as follows:

```
CALL stored_procedure_name(argument_list);
```

➤ **Drop Procedure statement**

The drop procedure statement removes a stored procedure. The following illustrates the syntax of the drop procedure statement:

```
DROP PROCEDURE procedure-name;
```

- To drop multiple stored procedures, you specify a comma-list of stored procedure names after the drop procedure keyword like this:

```
DROP PROCEDURE procedure-name1, procedure-name2...;
```

Example 1 find minimum number out of 2 number using Procedure.

```
Create or replace procedure
min (x in int, y in int, z inout int)
as $$
begin
    if x < y then
        z:= x;
    else
        z:= y;
    end if;
end;
$$
language plpgsql;
```

Output: call min (5,3,0)

	z integer	🔒
1		3

Example 2 Code for getting selective table details in output.

```
Create table emp (empid int, ename varchar(50), age int, salary numeric);  
INSERT INTO emp VALUES (1,'Mahesh',20,20000),  
(2,'Suresh',22,22000), (3,'Naresh',25,25000);  
Select * from emp;
```

	empid integer	ename character varying	age integer	salary numeric
1	1	Mahesh	20	20000
2	2	Suresh	22	22000
3	3	Naresh	25	25000

```
Create or replace procedure get_emp_detail (id int)  
as $$  
declare  
    emp_name emp.ename%type;  
    emp_sal emp.salary%type;  
begin  
    SELECT ename,salary INTO emp_name,emp_sal FROM emp  
    WHERE empid=id;  
    raise notice 'empid:%, ename:%, salary:%',  
    id,emp_name,emp_sal;  
end;  
$$  
language plpgsql;
```

Output: CALL get_emp_detail(1);

NOTICE: empid:1, ename:Mahesh, salary:20000 CALL

Example 3 Code for increasing salary by 10 % of employee whose age is 25.

```
create or replace procedure increment (a in int) as $$  
declare  
    emp_name emp.ename%type;  
    emp_sal emp.salary%type;  
    newsalary int;  
Begin  
    SELECT ename,salary INTO emp_name,emp_sal FROM emp  
    WHERE age=a;  
    newsalary:=emp_sal+0.1*emp_sal;
```



```
UPDATE emp SET salary=newsalary WHERE age=a;  
raise notice 'newsalary:%',newsalary;  
end;  
$$  
language plpgsql;
```

NOTICE: newsalary:27500
CALL

Output: CALL increment (25);

Database Triggers

- A PostgreSQL trigger is a function invoked automatically whenever an event associated with a table occurs.
- A trigger is a special user-defined function associated with a table.
- To create a new trigger, you define a trigger function first, and then bind this trigger function to a table.
- Triggers are stored programs, which are automatically executed or fired when some event occurs.
- The difference between a trigger and a user-defined function is that a trigger is automatically invoked when a triggering event occurs.

➤ A trigger is executed due to the following circumstances listed below:

1. By a DDL (Data Definition Language) statement like DROP, ALTER, or CREATE.
 2. By a DML (Data Manipulation Language) statement like UPDATE, INSERT, or DELETE.
 3. By a database action like SHUTDOWN, STARTUP, LOGOFF, and LOGON.
- A trigger can be set on a schema, view, or database that has an event attached.

➤ Advantages of Triggers

- Generating some derived column values automatically
- Enforcing referential integrity
- Event logging and storing information on table access
- Auditing
- Synchronous replication of tables
- Imposing security authorizations
- Preventing invalid transactions

➤ Usages of Triggers

- Prevents improper transactions.
- Accumulates information on table usage.
- Monitor critical information.

➤ Types of Triggers In PL/pgSQL

- The triggers can be categorized based on parameters. Types are listed below:

1) Categorization on the trigger level.

1. **ROW Level trigger:** It gets executed for each record that got updated by a DML statement.
2. **STATEMENT Level trigger:** It gets executed only once by the event statement.

2) Categorization of the trigger timing.

1. **BEFORE trigger:** It gets executed prior to the specific event that has taken place.
2. **AFTER trigger:** It gets executed post the specific event that has taken place.

3) Categorization of the trigger event.

1. **DML trigger:** It gets executed if a DML event like an UPDATE, INSERT, or DELETE is performed.
2. **DDL trigger:** It gets executed if a DDL event like a DROP, ALTER, or CREATE is performed.
3. **DATABASE trigger:** It gets executed if a database event like SHUTDOWN, STARTUP, LOGOFF, and LOGON has taken place.

Note: Unlike other databases, a trigger is broken into two pieces

1. Trigger
2. Trigger Function

➤ CREATE TRIGGER Statement

- The CREATE TRIGGER statement creates a new trigger.

Syntax:

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER}
{INSERT [OR] | UPDATE [OR] | DELETE} ON table_name
[FOR [EACH] {ROW | STATEMENT}]
[WHEN (condition)]
EXECUTE PROCEDURE trigger_function_name
```

- In this syntax:
 - First, specify the name of the trigger after the TRIGGER keywords.
 - Second, specify the timing that cause the trigger to fire. It can be BEFORE or AFTER an event occurs.
 - Third, specify the event that invokes the trigger. The event can be INSERT , DELETE, UPDATE or TRUNCATE.
 - Fourth, specify the name of the table associated with the trigger after the ON keyword.
 - Fifth, specify the type of triggers which can be:

- Row-level trigger that is specified by the FOR EACH ROW clause.
- Statement-level trigger that is specified by the FOR EACH STATEMENT clause.
- A row-level trigger is fired for each row while a statement-level trigger is fired for each transaction.

➤ **TRIGGER Function**

```
CREATE FUNCTION trigger_function_name RETURNS trigger AS
$$
BEGIN
    --Statements
    RETURN NEW;
END;
$$ language plpgsql;
```

➤ **DROP TRIGGER statement**

- To delete a trigger from a table, you use the DROP TRIGGER statement with the following syntax:

```
DROP TRIGGER [IF EXISTS] trigger_name ON table_name;
```

Example 1 Write a trigger for insertion of a row into a person table. On firing the trigger, operation should be displayed.

- First, Create a table person.

```
create table person (id int, name varchar(100));
```

- Second, Create a function.

```
create or replace function user_value() returns trigger
as $$
```

```
begin
```

```
    if new.id<0 then
```

```
        raise 'less than zero value not allowed';
```

```
    end if;
```

```
    return New;
```

```
end;
```

```
$$
```

```
language plpgsql;
```

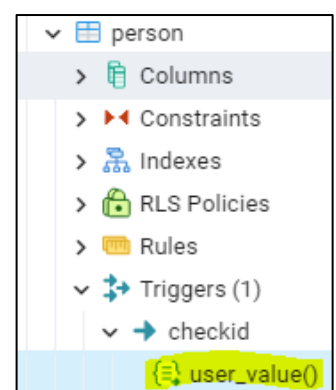
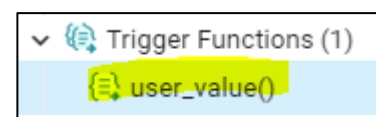
- At last, create a trigger.

```
create trigger checkid
```

```
before insert on person
```

```
for each row
```

```
execute procedure user_value();
```





- Now insert the value in person table:

```
insert into person values (-1,'Raj');
```

```
NOTICE: less than zero value not allowed
```

or

```
ERROR: less than zero value not allowed  
CONTEXT: PL/pgSQL function fun1() line 3 at RAISE  
SQL state: P0001
```

```
insert into person values (1,'Raj');
```

```
INSERT 0 1
```

```
Select * from person;
```

	id integer	name character varying
1	1	Raj

Question Bank Solution

Write a PL/SQL block to print the sum of even numbers from 1 to 100.

```
do $$  
declare  
    a int :=2;  
    sum int :=0;  
begin  
    while a <101 loop  
        sum := sum + a;  
        a = a+2;  
    end loop;  
    raise notice 'sum = %', sum;  
end;  
$$
```

Write a PL/SQL block to print the sum of odd numbers from 1 to 100.

```
do $$  
declare  
    a int :=1;  
    sum int :=0;  
begin  
    loop
```



```
        sum = sum + a;  
        a = a+ 2;  
        exit when a > 100;  
    end loop;  
    raise notice 'sum = %', sum;  
end;  
$$
```

Define PL/SQL. Write PL/SQL block to print sum of even numbers between 1 to 20.

```
do $$  
declare  
    a int :=2;  
    sum int :=0;  
begin  
    while a <21 loop  
        sum := sum + a;  
        a = a+2;  
    end loop;  
    raise notice 'sum = %', sum;  
end;  
$$
```

Write A PL/SQL block to print the sum of Numbers from 1 to 50.

```
do $$  
declare  
    a int;  
    sum int :=0;  
begin  
    for a in 1..50 loop  
        sum = sum + a;  
    end loop;  
    raise notice 'sum = %', sum;  
end;  
$$
```



Write A PL/SQL block to print the given number is Odd or Even

```
do $$
declare
    a int :=5;
begin
    if mod (a,2)=0 then
        raise notice 'even';
    else
        raise notice 'odd';
    end if;
end;
$$
```

Write a PL/SQL cursor to increase the salary by 10% for the employees having salary less then 700000.

```
Do $$
DECLARE
    C1 employee.employee_id%TYPE;
    C2 employee.salary%TYPE;
    cur CURSOR FOR SELECT employee_id, salary FROM
    employees WHERE salary < 70000;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO C1,C2;
        EXIT WHEN NOT FOUND;
        UPDATE employees SET salary = salary + (0.1*
        salary) WHERE current of cur;
    END LOOP;
    CLOSE cur;
END;
$$
```



Write a PL/SQL cursor to display the names and branch of all students from the STUDENT relation.

```
do $$
declare
    c1_name std.name%type;
    c2_branch std.branch%type;
    s_cursor cursor for select name,branch from std;
begin
    open s_cursor;
    loop
        fetch s_cursor into c1_name,c2_branch;
        exit when not found;
        raise notice 'name:%,branch:
        %',c1_name,c2_branch;
    end loop;
    close s_cursor;
end;
$$
```

Write a PL/SQL block using explicit cursor that will display the customer name, the fixed deposit number and the fixed deposit amount of the first 5 customers holding the highest amount in fixed deposits.

Use following database:

cust_mstr (custno, name, occupation)

fd_dtls (fd_ser_no, custno, fd_no, type, period, opndt, duedt, amt, dueamt)

acct_fd_cust_dtls (acct_fd_no, custno)

```
do $$
declare
    c1_cust_no cust_mstr.custno%type;
    c2_name cust_mstr.name%type;
    c3_fd_no fd_dtls.fd_no%type;
    c4_amount fd_dtls.amt%type;
    c_cursor cursor for select name,fd_no,amt from
    cust_mstr,fd_dtls where cust_mstr.custno =
    fd_dtls.custno order by amt desc limit 5;
begin
    open c_cursor;
    loop
```

```
        fetch c_cursor into c2_name, c3_fd_no,
        c4_amount;
        exit when not found;
        raise notice 'name:%,fd_no:%,amount:%',
        c2_name,c3_fd_no,c4_amount;
    end loop;
    close c_cursor;
end;
$$
```

A stored function is created to perform the acct_no check operation. f_ChkAcctNo() is the name of function which accepts a variable acct_no from the user and returns value 0 if acct_no does not exist or 1 if acct_no exists.

```
create or replace function f_ChkAcctNo(acct_no int)
returns int as $$
declare
    dummy_ano int;
begin
    select ano into dummy_ano from account_qb where ano
    = acct_no;
    if dummy_ano = acct_no then
        return 1;
    else
        return 0;
    end if;
end;
$$
language plpgsql;
```

Write a PL/SQL program for inserting even numbers in EVEN table and odd number in ODD table from number 1 to 50.

```
do $$
declare
    a int;
begin
    for a in 1..50 loop
        if mod(a,2)=0 then
```

```
        insert into even values (a);
    else
        insert into odd values (a);
    end if;
end loop;
raise notice 'update done';
end;
$$
```

Write PL/SQL block to print whether the given number is Armstrong number or not.

```
do $$
declare
    a int :=153; sum int :=0; d int; temp int;
begin
    temp :=a;
    while temp <> 0 loop
        d := mod (temp,10);
        sum := sum + (d*d*d);
        temp:= (temp/10);
    end loop;
    if sum = a then
        raise notice 'armstrong number';
    else
        raise notice 'not armstrong number';
    end if;
end;$$
```

Write a Stored procedure to Insert Data in

Student_Detail [Student_id, Name, Age, Class] table.

```
create table Student_Detail (Student_id int,
Name varchar (100), Age int, Class varchar(10));
create or replace procedure
inser_std_info (s_id Student_Detail.Student_id %type,
s_name Student_Detail.Name%type,
s_age Student_Detail.Age%type,
s_class Student_Detail.Class%type) as $$
```



```
begin
    insert into Student_Detail
    values (s_id,s_name,s_age,s_class);
end;
$$language plpgsql;
call inser_std_info(1,'abc',15,'d10');
```

Write a PL/SQL code to count the number of Sundays between the two inputted dates.

```
CREATE OR REPLACE FUNCTION count_sundays(start_date
DATE, end_date DATE) RETURNS INTEGER AS $$
DECLARE
    num_sundays INTEGER := 0;
    curr_date DATE := start_date;
BEGIN
    WHILE curr_date <= end_date LOOP
        IF EXTRACT(DOW FROM curr_date) = 0 THEN
            -- 0 represents Sunday
            num_sundays := num_sundays + 1;
        END IF;
        curr_date := curr_date + 1;
    END LOOP;
    RETURN num_sundays;
END;
$$ LANGUAGE plpgsql;
SELECT count_sundays('2026-08-01', '2026-08-31');
```

Write PL/SQL code block to increment the employee's salary by 1000 whose employee id is 102 from the given table below. Schema emp1 (eid, salary)

```
do $$
declare
    c1_eid emp1.eid%type;
begin
    c1_eid := 1;
    if c1_eid = 1 then
        update emp1 set salary = salary +1000 where eid=1;
    end if;
```



```
raise notice 'updated';  
end;  
$$
```

Write a PL/SQL code to find whether a given string is palindrome or not.

```
do $$  
declare  
    string text := 'bob';  
    reverse_string text;  
begin  
    reverse_string := reverse (string);  
    if string = reverse_string then  
        raise notice 'The string is Palindrome';  
    else  
        raise notice 'The string is not Palindrome';  
    end if;  
end; $$
```

Write PL/SQL program to find the sum of digits of a number.

```
do $$  
declare  
    a int :=123;  
    sum int := 0;  
    digit int;  
begin  
    while a <> 0 loop  
        digit = mod (a,10);  
        sum = sum + digit;  
        a := (a/10);  
    end loop;  
    raise notice 'ans:%',sum;  
end; $$
```

Write a trigger for insertion of a row into supplier table. On executing the trigger sid less than zero should not be allowed. The schema is Supplier (sid, sname, city)Also show by inserting value in the table.

```
CREATE TABLE Supplier (sid INTEGER PRIMARY KEY,  
                        sname VARCHAR,city VARCHAR);
```



```
CREATE OR REPLACE FUNCTION check_sid()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.sid < 0 THEN  
        RAISE EXCEPTION 'sid cannot be less than zero';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER before_insert_supplier  
BEFORE INSERT ON Supplier  
FOR EACH ROW  
EXECUTE Procedure check_sid();
```

```
INSERT INTO Supplier (sid, sname, city) VALUES (1,  
'Supplier A', 'City A');
```

INSERT 0 1

Query returned successfully in 83 msec.

```
INSERT INTO Supplier (sid, sname, city) VALUES (-1,  
'Supplier B', 'City B');
```

ERROR: sid cannot be less than zero

CONTEXT: PL/pgSQL function check_sid() line 4 at RAISE

SQL state: P0001

Create a function using PL/pgSQL to find total number of customers in the customers table.

Customers

Id	Name	Department	Salary
1	alex	web developer	35000
2	ricky	program developer	45000
3	mohan	web designer	35000

Also call the function at the end.

```
CREATE TABLE Customers ( Id INTEGER PRIMARY KEY, Name  
VARCHAR(100), Department VARCHAR(100), Salary INTEGER );
```



```
INSERT INTO Customers VALUES
(1, 'alex', 'web developer', 35000),
(2, 'ricky', 'program developer', 45000),
(3, 'mohan', 'web designer', 35000);
```

```
CREATE OR REPLACE FUNCTION count_customers ()
RETURNS INTEGER AS $$
DECLARE
    total_customers INTEGER;
BEGIN
    SELECT COUNT(*) INTO total_customers FROM Customers;
    RETURN total_customers;
END;
$$ LANGUAGE plpgsql;

--Call function to get the output
SELECT count_customers();
```

Write a PL/SQL block to create a stored procedure named transfer that transfers a specified amount of money from one account to another.

```
CREATE TABLE accounts (id int, name varchar (100),
balance numeric, primary key(id));
INSERT INTO accounts
VALUES (1, 'Bob',10000), (2, 'Alice',10000);
```

```
SELECT * FROM accounts;
```

id	name	balance
1	Bob	10000
2	Alice	10000

```
CREATE or REPLACE PROCEDURE transfer (sender int, receiver
int, amount int )
language plpgsql AS $$
BEGIN
    -- subtracting the amount from the sender's account
    UPDATE accounts
    SET balance = balance - amount WHERE id = sender;
```



```
-- adding the amount to the receiver's account
UPDATE accounts
SET balance = balance + amount WHERE id = receiver;
COMMIT;
END;$$;
```

```
--Calling a stored procedure
CALL transfer (1,2,1000);
```

```
SELECT * FROM accounts;
id | name  | balance
----+-----+-----
1  | Bob   | 9000
2  | Alice | 11000
```

Write a PL/pgSQL program function using WHILE loop for calculating the average of the numbers entered by in array in postgresql.

```
CREATE OR REPLACE FUNCTION calculate_average(numbers
INTEGER[])
RETURNS NUMERIC AS $$
DECLARE
    sum NUMERIC := 0;
    i INTEGER := 1;
    n INTEGER := array_length(numbers, 1);
BEGIN
    -- Loop through each element in the array
    WHILE i <= n LOOP
        sum := sum + numbers[i];
        i := i + 1;
    END loop;
    -- Calculate and return the average
    RETURN sum / n;
END;
$$ LANGUAGE plpgsql;
```

	calculate_average	
	numeric	🔒
1	31.4000000000000000	

```
SELECT calculate_average(ARRAY[10, 27, 30, 40, 50]);
```



Use the following table:

Stock (item_id, item_name, units_in_stock, reorder_level)

Write a PL/pgSQL trigger function and a trigger that enforces the following rules before inserting a new row into the Stock table:

1. The units_in_stock value must be greater than zero.
2. The reorder_level must not be negative and must not exceed the units_in_stock.

If either condition is violated, the insertion should be rejected with a meaningful error message:

- If units_in_stock is invalid, display: 'Units in stock cannot be negative or zero'
- If reorder_level is invalid, display: 'Reorder level cannot be negative or greater than units in stock'

Also show by inserting invalid values in the table.

-- Create Table

```
CREATE TABLE Stock ( item_id INT, item_name VARCHAR(50),  
units_in_stock INT, reorder_level INT);
```

-- Trigger Function

```
CREATE OR REPLACE FUNCTION check_stock_values()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    IF NEW.units_in_stock <= 0 THEN
```

```
        RAISE EXCEPTION 'Units in stock cannot be negative or  
zero';
```

```
    END IF;
```

```
    IF NEW.reorder_level < 0 OR NEW.reorder_level >
```

```
    NEW.units_in_stock THEN
```

```
        RAISE EXCEPTION
```

```
        'Reorder level cannot be negative or greater than  
units in stock';
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END; $$
```

```
LANGUAGE plpgsql;
```

-- Trigger

```
CREATE TRIGGER trg_check_stock BEFORE INSERT ON Stock
```

```
FOR EACH ROW EXECUTE FUNCTION check_stock_values();
```

```
INSERT INTO Stock VALUES (1, 'Pen', 100, 20);
```



-- Row inserted successfully

```
INSERT INTO Stock VALUES (2, 'Pencil', 0, 10);
```

ERROR: Units in stock cannot be negative or zero

```
INSERT INTO Stock VALUES (3, 'Book', 50, -5);
```

ERROR: Reorder level cannot be negative or greater than units in stock

```
INSERT INTO Stock VALUES (4, 'Eraser', 30, 40);
```

ERROR: Reorder level cannot be negative or greater than units in stock

Given the existing tables EMP and EMP_LOG, write a PL/PGSQL trigger that will be executed whenever an UPDATE query is applied to the EMP table. This trigger should insert the corresponding values into the EMP_LOG table.

emp_id	emp_name	emp_salary	emp_dept
[PK] integer	character varying (100)	numeric (10,2)	character varying (100)

log_id	emp_id	emp_name	old_emp_salary	old_emp_dept	new_emp_salary	new_emp_dept	update_timestamp
[PK] integer	integer	character varying (100)	numeric (10,2)	character varying (100)	numeric (10,2)	character varying (100)	timestamp without time zone

CREATE TABLE EMP

```
(emp_id INTEGER PRIMARY KEY, emp_name VARCHAR(100),  
emp_salary NUMERIC(10,2), emp_dept VARCHAR(100) );
```

CREATE TABLE EMP_LOG

```
(log_id INTEGER PRIMARY KEY, emp_id INTEGER,  
emp_name VARCHAR(100), old_emp_salary NUMERIC(10,2),  
old_emp_dept VARCHAR(100), new_emp_salary NUMERIC(10,2),  
new_emp_dept VARCHAR(100), update_timestamp TIMESTAMP );
```

INSERT INTO EMP VALUES

```
(101, 'Ajay', 50000, 'HR'),  
(102, 'Vijay', 60000, 'IT'),  
(103, 'Neha', 55000, 'Finance');
```



```
CREATE OR REPLACE FUNCTION emp_log_fn()
```

```
RETURNS TRIGGER AS
```

```
$$
```

```
BEGIN
```

```
    IF OLD.emp_salary <> NEW.emp_salary
```

```
    OR OLD.emp_dept <> NEW.emp_dept THEN
```

```
        INSERT INTO emp_log VALUES ( Old.emp_id, Old.emp_id,  
Old.emp_name, OLD.emp_salary, OLD.emp_dept,  
NEW.emp_salary, NEW.emp_dept, Current_timestamp );
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$
```

```
LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_emp_log
```

```
AFTER UPDATE ON emp
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION emp_log_fn();
```

```
UPDATE EMP SET emp_salary = 65000, emp_dept = 'Marketing'  
WHERE emp_id = 101;
```

```
SELECT * FROM EMP;
```

	emp_id [PK] integer	emp_name character varying (100)	emp_salary numeric (10,2)	emp_dept character varying (100)
1	102	Vijay	60000.00	IT
2	103	Neha	55000.00	Finance
3	101	Ajay	65000.00	Marketing

```
SELECT * FROM EMP_LOG;
```

	log_id [PK] integer	emp_id integer	emp_name character varying (100)	old_emp_salary numeric (10,2)	old_emp_dept character varying (100)	new_emp_salary numeric (10,2)	new_emp_dept character varying (100)	update_timestamp timestamp without time zone
1	101	101	Ajay	50000.00	HR	65000.00	Marketing	2026-07-10 22:43:55.9...



Write a PL/pgSQL procedure for transaction management system that includes the department table and transaction table with details as mentioned below
Departments(dept_id, dept_name, balance) and transactions (transaction_id, from_dept_id, to_dept_id, amount, transaction_date)

```
CREATE TABLE departments
```

```
(dept_id INT PRIMARY KEY, dept_name VARCHAR(20), balance  
NUMERIC(10,2) );
```

```
CREATE TABLE transactions
```

```
(transaction_id INT PRIMARY KEY, from_dept_id INT,  
to_dept_id INT, amount NUMERIC(10,2), transaction_date  
DATE );
```

```
INSERT INTO departments VALUES
```

```
(1, 'HR', 50000),  
(2, 'IT', 75000),  
(3, 'Finance', 100000);
```

```
CREATE OR REPLACE PROCEDURE transfer_amount
```

```
(tid int, from_id INT, to_id INT, amt NUMERIC)
```

```
LANGUAGE plpgsql AS
```

```
$$
```

```
BEGIN
```

```
-- Deduct amount from source department
```

```
UPDATE departments SET balance = balance - amt WHERE  
dept_id = from_id;
```

```
-- Add amount to destination department
```

```
UPDATE departments SET balance = balance + amt WHERE  
dept_id = to_id;
```

```
-- Store transaction details
```

```
INSERT INTO transactions VALUES
```

```
(tid, from_id, to_id, amt, CURRENT_DATE );
```

```
RAISE NOTICE 'Transaction Completed';
```

```
END;
```

```
$$;
```

```
CALL transfer_amount(101,1,2,10000);
```

SELECT * FROM departments;

	dept_id [PK] integer	dept_name character varying (20)	balance numeric (10,2)
1	3	Finance	100000.00
2	1	HR	40000.00
3	2	IT	85000.00

SELECT * FROM transactions;

	transaction_id [PK] integer	from_dept_id integer	to_dept_id integer	amount numeric (10,2)	transaction_date date
1	101	1	2	10000.00	2026-07-10

Write a pl-pgsql code to find whether given number is perfect square or not using if...else statement.

```
DO
$$
DECLARE
    n INTEGER := 49;    -- Change the number here
    root INTEGER;
BEGIN
    root := SQRT(n);
    IF root * root = n THEN
        RAISE NOTICE '% is a Perfect Square', n;
    ELSE
        RAISE NOTICE '% is Not a Perfect Square', n;
    END IF;
END;
$;$
```